LAPORAN TUGAS PRAKTIKUM 9&10

FRAMEWORK PEMROGRAMAN WEB

Dosen Pengampu : Kamal Prihandani, S.Kom., M.Kom.

NAMA : Refi Abdillah

NPM : 2410631170045

KELAS : 5C Informatika

INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS SINGAPERBANGSA KARAWANG

2026

PRAKTIKUM 9

Pada tahap awal praktikum ini, saya membuat kelas Form Request bernama `UpdateProductRequest` menggunakan perintah Artisan Laravel. Pembuatan Form Request terpisah bertujuan untuk menerapkan prinsip *Single Responsibility*, yaitu memisahkan logika validasi input dari controller agar kode tetap bersih dan mudah dipelihara.

```
PS C:\laragon\www\laravel12> php artisan make:request UpdateProductRequest

   INFO  Request [app/Http/Requests/UpdateProductRequest.php] created successfully.
```

Selanjutnya, saya mengonfigurasi method `authorize()` dan `rules()` pada `UpdateProductRequest.php`. Method `authorize()` memverifikasi bahwa hanya pengguna dengan role admin yang diizinkan memperbarui data produk.

app/Http/Requests/UpdateProductRequest.php

```php
12      public function authorize(): bool
13      {
14          return auth()->user()?->role === 'admin';
15      }
16
17      public function rules(): array
18      {
19          $productId = is_object($this->product) ? $this->product->id : ($this->product ?? $this->route('
20
21          return [
22              'category_id' => 'required|exists:categories,id',
23              'code' => 'required|string|max:20|unique:products,code,' . $productId,
24              'name' => 'required|string|max:150',
25              'unit' => 'required|string|max:20',
26              'price' => 'required|integer|min:0',
27              'stock' => 'required|integer|min:0',
28          ];
29      }
30
31      public function messages(): array
32      {
33          return [
34              'code.unique' => 'Kode produk sudah digunakan, gunakan kode lain.',
```

Pada aturan validasi, penambahan klausa `unique:products,code,' . $productId` memberikan pengecualian terhadap baris milik produk yang sedang diedit. Dengan pengecualian ini, menyimpan perubahan tanpa mengganti kode produk tidak akan memicu validasi duplikasi.

Pada bagian ini, saya mengimplementasikan Route Model Binding pada method `edit()`, `update()`, dan `destroy()` di dalam `ProductController.php`. Dengan menyertakan tipe model `Product $product` pada parameter, Laravel secara otomatis mencocokkan nilai parameter `{product}` di URL dan menyuntikkan instance model produk terkait tanpa perlu menuliskan query pencarian manual seperti `Product::findOrFail($id)`.

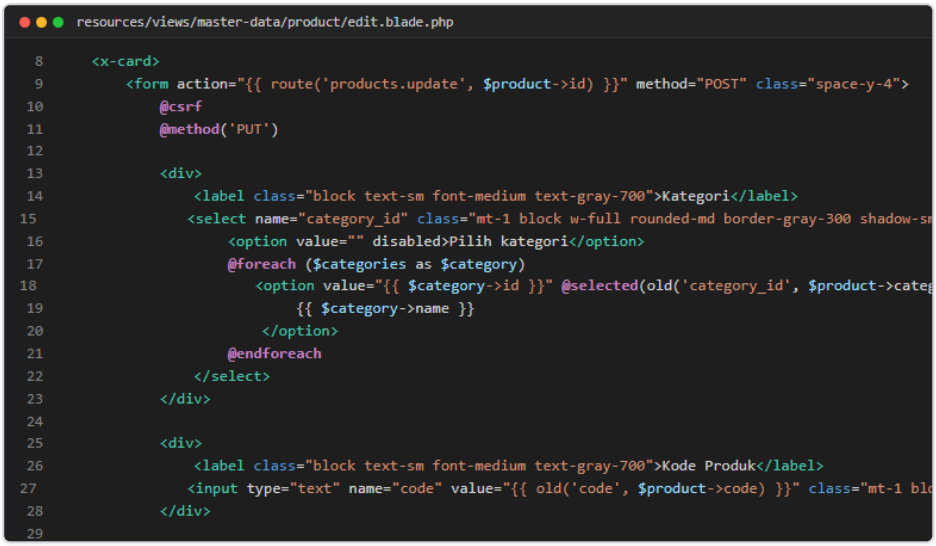
app/Http/Controllers/ProductController.php

```
49      public function edit(Product $product)
50      {
51          $categories = Category::orderBy('name')->get();
52          return view('master-data.product.edit', compact('product', 'categories'));
53      }
54
55      public function update(UpdateProductRequest $request, Product $product)
56      {
57          $product->update($request->validated());
58
59          return redirect()->route('products.index')->with('success', 'Produk berhasil diperbarui.');
60      }
61
62      public function destroy(Product $product)
63      {
64          $product->delete();
65
66          return redirect()->route('products.index')->with('success', 'Produk berhasil dihapus.');
```

Pada method `update()`, proses pembaruan data dilakukan melalui `$product->update($request->validated())` yang secara otomatis mengeksekusi query SQL UPDATE hanya pada data yang telah divalidasi. Setelah berhasil, pengguna diarahkan kembali ke `products.index` disertai pesan sukses pada flash session.

Pada method `destroy()`, baris data produk dihapus dari basis data menggunakan `$product->delete()`, kemudian pengguna dialihkan kembali ke daftar produk beserta pesan pemberitahuan bahwa produk telah berhasil dihapus.

Pada tahap ini, saya membuat tampilan form ubah produk pada file `resources/views/master-data/product/edit.blade.php`. Form ini diarahkan ke route `products.update` dengan menyertakan parameter objek `$product->id`.



Karena formulir HTML secara baku hanya mendukung metode pengiriman GET dan POST, saya menyematkan directive `@method('PUT')` untuk melakukan *method spoofing*. Directive ini menyisipkan input tersembunyi `_method=PUT` sehingga Laravel dapat memetakan permintaan ke rute update yang menggunakan verb PUT/PATCH sesuai standar RESTful API.

Setiap elemen input diisi menggunakan helper `old('field', $product->field)`. Fungsi ini memastikan data awal produk langsung terisi saat form pertama kali dibuka, dan apabila terjadi kesalahan validasi, nilai inputan terakhir pengguna tetap dipertahankan pada formulir.

Pada tahap ini, saya menambahkan tombol aksi edit dan hapus pada setiap baris data di `resources/views/master-data/product/index.blade.php`. Tombol hapus dibungkus dalam tag form dengan directive `@method('DELETE')`.

resources/views/master-data/product/index.blade.php (kolom aksi & konfirmasi)

```
81          <td class="px-4 py-2 text-sm text-center">
82              <div class="flex justify-center items-center space-x-2">
83                  <a href="{{ route('products.edit', $product->id) }}"
84                     class="bg-yellow-500 hover:bg-yellow-600 text-white px-3 py-1 rounded text-xs font-
85                      Edit
86                  </a>
87
88                  <form action="{{ route('products.destroy', $product->id) }}" method="POST"
89                      class="inline"
90                      onsubmit="return confirm('Apakah Anda yakin ingin menghapus produk ini?');">
91                      @csrf
92                      @method('DELETE')
93                      <button type="submit"
94                         class="bg-red-600 hover:bg-red-700 text-white px-3 py-1 rounded text-xs font-ser
95                          Hapus
```

Untuk mengantisipasi penghapusan data secara tidak sengaja oleh pengguna, saya menyematkan atribut JavaScript `onsubmit="return confirm('Apakah Anda yakin ingin menghapus produk ini?');"`. Saat tombol hapus ditekan, kotak dialog konfirmasi pop-up akan muncul meminta persetujuan pengguna sebelum permintaan dikirim ke server.

app/Http/Controllers/CategoryController.php (Latihan 1 & 2)

```
40      public function update(Request $request, Category $category)
41      {
42          $validated = $request->validate([
43              'name' => 'required|string|max:100|unique:categories,name,' . $category->id,
44              'description' => 'nullable|string|max:255',
45          ]);
46
47          $category->update($validated);
48
49          return redirect()->route('categories.index')->with('success', 'Kategori berhasil diperbarui.');
50      }
51
52      public function destroy(Category $category)
53      {
54          "color: #6a9955;">// Validasi agar kategori yang masih memiliki produk tidak dapat dihapus
55          if ($category->products()->exists()) {
56              return redirect()->route('categories.index')->with('error', 'Kategori tidak dapat dihapus ka
57          }
58
59          $category->delete();
```

Pada latihan modul Pertemuan 9, saya menerapkan pola Route Model Binding serupa pada `CategoryController.php` untuk operasi `update()` dan `destroy()`. Pada method `destroy()`, saya menambahkan validasi integritas relasi: jika kategori masih memiliki produk terkait (`$category->products()->exists()`), proses penghapusan dicegah dan pesan error ditampilkan demi menjaga integritas data.

PRAKTIKUM 10

Pada praktikum pertemuan 10 ini, saya memodifikasi method `index()` pada `ProductController.php` agar dapat menerima input pencarian kata kunci dan filter kategori melalui query parameter URL. Fitur ini dirancang untuk mempermudah kasir dan admin dalam menemukan produk secara cepat dan efisien di toko.

app/Http/Controllers/ProductController.php (method index)

```php
27    public function index(Request $request)
28    {
29        $query = Product::with('category');
30
31        if ($request->filled('search')) {
32            $keyword = $request->search;
33            $query->where(function ($q) use ($keyword) {
34                $q->where('name', 'like', "%{$keyword}%")
35                    ->orWhere('code', 'like', "%{$keyword}%");
36            });
37        }
38
39        if ($request->filled('category_id')) {
40            $query->where('category_id', $request->category_id);
41        }
42
43        if ($request->filled('stock_status')) {
44            if ($request->stock_status === 'habis') {
45                $query->where('stock', 0);
46            } elseif ($request->stock_status === 'menipis') {
47                $query->where('stock', '>', 0)->where('stock', '<', 10);
48            } elseif ($request->stock_status === 'tersedia') {
```

Saya menggunakan method `$request->filled('search')` untuk memeriksa keberadaan parameter pencarian dan memastikan bahwa nilainya bukan string kosong. Kondisi pencarian nama dan kode produk dibungkus ke dalam sebuah closure function dengan klausul `where` dan `orWhere`. Pembungkusan ini sangat penting agar query SQL mengelompokkan kedua kondisi tersebut dalam tanda kurung `(name LIKE ? OR code LIKE ?)`, sehingga tidak bertabrakan dengan operator logika AND saat digabungkan dengan filter `category_id`.

Selain itu, saya juga menambahkan fitur filter status stok (Tersedia, Menipis, Habis) dan opsi penentuan jumlah data per halaman (`per_page`) sesuai dengan penugasan latihan pada modul Pertemuan 10.

Pada tahap ini, saya menambahkan komponen antarmuka pencarian dan dropdown kategori di atas tabel produk pada file `resources/views/master-data/product/index.blade.php`. Formulir ini menggunakan method GET agar parameter pencarian terlampir pada URL.

resources/views/master-data/product/index.blade.php (form search & filter)

```
32        <form method="GET" action="{{ route('products.index') }}" class="flex flex-wrap gap-3 mb-6 items
33            <input type="text" name="search" value="{{ request('search') }}"
34                placeholder="Cari nama atau kode produk..."
35                class="flex-1 min-w-[200px] rounded-md border-gray-300 shadow-sm text-sm">
36
37            <select name="category_id" class="rounded-md border-gray-300 shadow-sm text-sm">
38                <option value="">Semua Kategori</option>
39                @foreach ($categories as $category)
40                    <option value="{{ $category->id }}" @selected(request('category_id') == $category->i
41                        {{ $category->name }}
42                    </option>
43                @endforeach
44            </select>
45
46            <button type="submit" class="bg-indigo-600 hover:bg-indigo-700 text-white px-4 py-2 rounded-
```

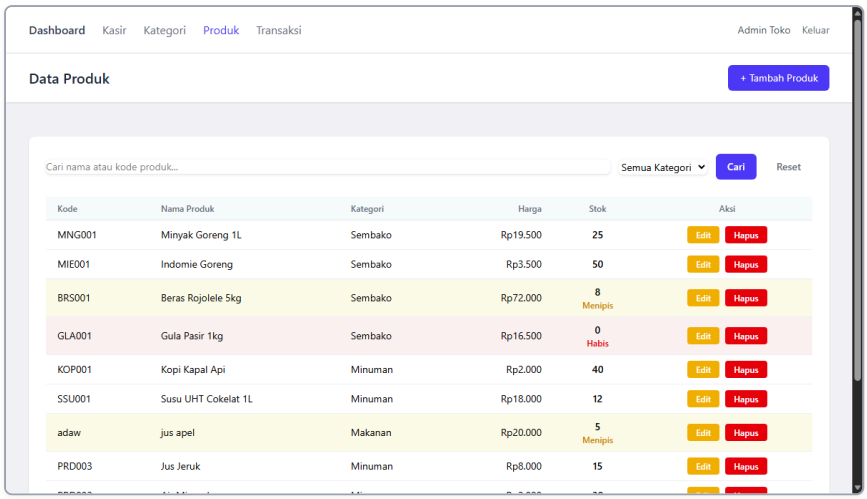
Pada kolom input teks pencarian, nilai kata kunci dipertahankan menggunakan `request('search')`. Pada elemen select kategori, saya menggunakan directive Blade `@selected(request('category_id') == $category->id)` agar pilihan kategori pengguna tetap terpilih secara otomatis setelah formulir dikirim. Tersedia pula tombol Reset yang mengarahkan kembali ke route `products.index` untuk membersihkan seluruh filter.

resources/views/master-data/product/index.blade.php (pagination links)

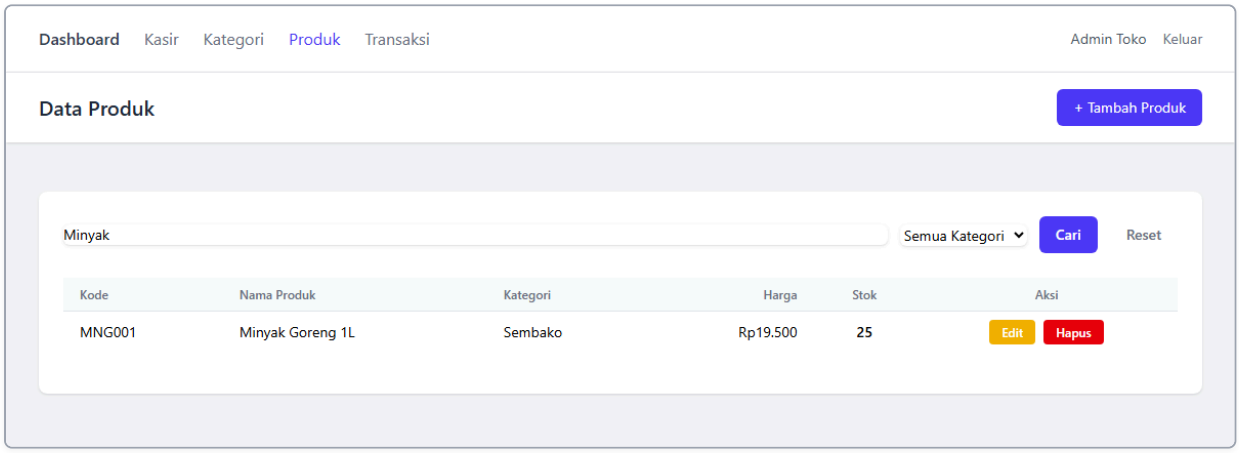
```
110        <div class="mt-4">
111            {{ $products->links() }}
112        </div>
```

Pada bagian bawah tabel, navigasi pagination ditampilkan menggunakan `{{ $products->links() }}`. Pada controller, pagination dipanggil dengan `paginate(10)->withQueryString()`. Fungsi `withQueryString()` berperan penting untuk mempertahankan parameter pencarian dan filter pada setiap tautan halaman. Tanpa method ini, saat pengguna berpindah halaman, parameter pencarian akan hilang dan sistem akan kembali menampilkan seluruh data tanpa filter.

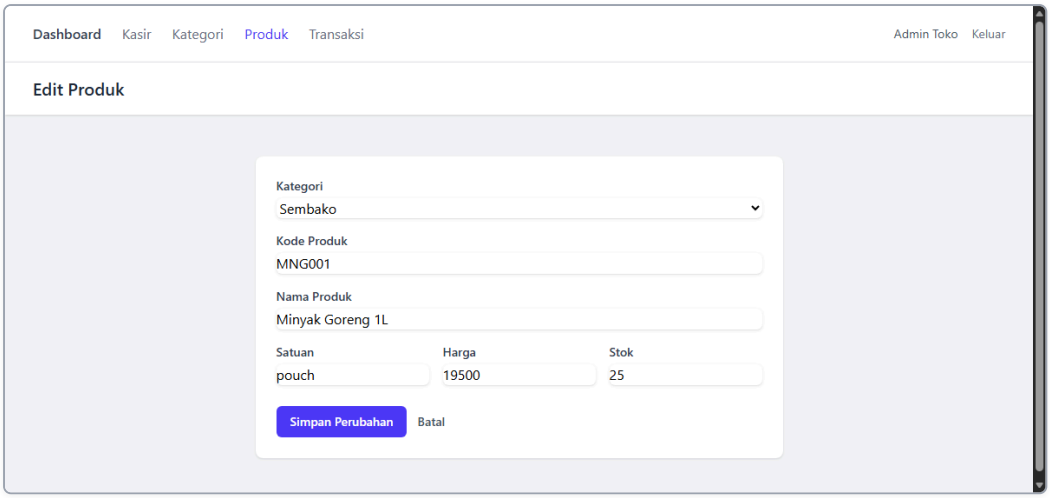
HASIL PENGUJIAN SISTEM DI WEB BROWSER



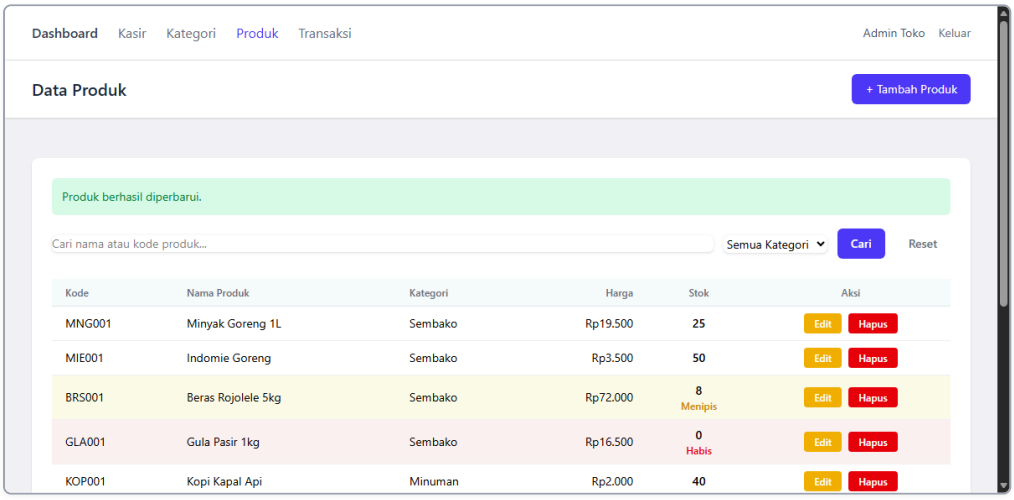
Gambar di atas menampilkan hasil implementasi antarmuka Data Produk pada aplikasi web POS Barokah Mart. Halaman ini memuat formulir pencarian produk, filter dropdown kategori, tabel daftar produk yang dilengkapi pewarnaan status stok (merah untuk stok habis dan kuning untuk stok menipis), tombol aksi Edit dan Hapus, serta komponen navigasi pagination di sudut kanan bawah tabel.



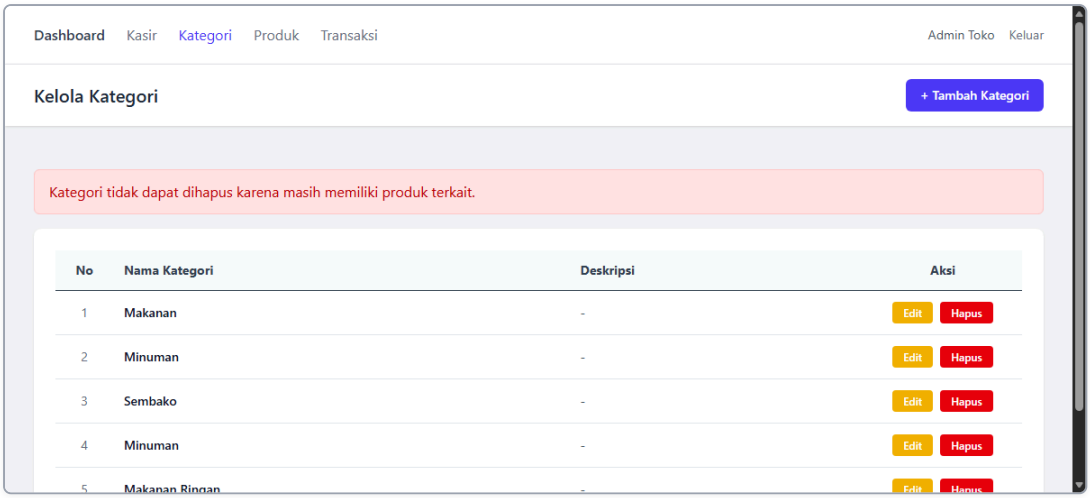
Gambar di atas menunjukkan hasil pengujian fitur pencarian produk dengan memasukkan kata kunci "Minyak". Sistem secara dinamis memproses query parameter pada URL (`/products?search=Minyak`) dan menyaring data sehingga hanya produk "Minyak Goreng 1L" yang tampil pada tabel secara akurat tanpa kendala.



Gambar di atas memperlihatkan antarmuka form Ubah Produk saat tombol Edit dipilih pada salah satu data produk. Berkat implementasi Route Model Binding pada controller dan fungsi `old('field', $product->field)` pada view, seluruh informasi awal produk seperti nama kategori, kode produk, nama produk, satuan, harga, dan stok otomatis terisi ke dalam masing-masing kolom formulir.



Gambar di atas menunjukkan respon sistem setelah pengguna memperbarui data produk dan menekan tombol "Simpan Perubahan". Sistem memvalidasi data menggunakan `UpdateProductRequest`, melakukan pembaruan pada database, lalu mengarahkan kembali ke halaman Data Produk disertai pesan flash session berwarna hijau bertuliskan "Produk berhasil diperbarui.".



Gambar di atas menampilkan hasil pengujian latihan Pertemuan 9 mengenai proteksi integritas data relasional pada halaman Kelola Kategori. Ketika pengguna berupaya menghapus kategori yang masih memiliki relasi dengan produk di dalam basis data (seperti kategori Makanan atau Minuman), sistem mendeteksi keberadaan data anak tersebut melalui `$category->products()->exists()` dan menggagalkan proses penghapusan.

Sistem kemudian menampilkan banner notifikasi error berwarna merah dengan pesan peringatan: *"Kategori tidak dapat dihapus karena masih memiliki produk terkait."*. Dengan mekanisme validasi ini, integritas referensial data transaksi dan produk di dalam basis data tetap terlindungi dari risiko data yatim (*orphan records*).